In [2]:
```python
import pandas as pd
```

In [3]:
```python
df = pd.read_csv("customer_churn_data.csv")
```

In [4]:
```python
df.head()
```

Out[4]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month	NaN	2208.32	No	Yes

In [5]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       1000 non-null   int64
 1   Age              1000 non-null   int64
 2   Gender           1000 non-null   object
 3   Tenure           1000 non-null   int64
 4   MonthlyCharges   1000 non-null   float64
 5   ContractType     1000 non-null   object
 6   InternetService  703 non-null    object
 7   TotalCharges     1000 non-null   float64
 8   TechSupport      1000 non-null   object
 9   Churn            1000 non-null   object
dtypes: float64(2), int64(3), object(5)
memory usage: 78.3+ KB
```

In [6]:
```python
df.isnull()
```

Out[6]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	True	False	False	False
...	...	...	...	...	...	...	...	...	...	...
995	False	False	False	False	False	False	False	False	False	False
996	False	False	False	False	False	False	True	False	False	False
997	False	False	False	False	False	False	False	False	False	False
998	False	False	False	False	False	False	True	False	False	False
999	False	False	False	False	False	False	True	False	False	False

1000 rows × 10 columns

In [7]:
```python
df.isna()
```

Out[7]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	True	False	False	False
...	...	...	...	...	...	...	...	...	...	...
995	False	False	False	False	False	False	False	False	False	False
996	False	False	False	False	False	False	True	False	False	False
997	False	False	False	False	False	False	False	False	False	False
998	False	False	False	False	False	False	True	False	False	False
999	False	False	False	False	False	False	True	False	False	False

1000 rows × 10 columns

In [8]:
```python
df.isna().sum()
```

Out[8]:
```
CustomerID           0
Age                  0
Gender               0
Tenure               0
MonthlyCharges       0
ContractType         0
InternetService    297
TotalCharges         0
TechSupport          0
Churn                0
dtype: int64
```

In [9]:
```python
df.isna().sum().sum()
```

Out[9]:
```
np.int64(297)
```

In [11]:
```python
df.isna().sum().sum()
```

Out[11]:
```
np.int64(297)
```

In [12]:
```python
df["InternetService"] = df["InternetService"].fillna(" ")
```

In [13]:
```python
df.isna().sum()
```

Out[13]:
```
CustomerID         0
Age                0
Gender             0
Tenure             0
MonthlyCharges     0
ContractType       0
InternetService    0
TotalCharges       0
TechSupport        0
Churn              0
dtype: int64
```

In [14]:
```python
df.head()
```

Out[14]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month		2208.32	No	Yes

In [15]:
```python
df.duplicated()
```

Out[15]:
```
0      False
1      False
2      False
3      False
4      False
       ...
995    False
996    False
997    False
998    False
999    False
Length: 1000, dtype: bool
```

In [16]:
```python
df.duplicated().sum()
```

Out[16]:
```
np.int64(0)
```

In [17]:
```python
df.head()
```

Out[17]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month		2208.32	No	Yes

In [18]:
```python
df.describe()
```

Out[18]:

	CustomerID	Age	Tenure	MonthlyCharges	TotalCharges
count	1000.000000	1000.000000	1000.00000	1000.000000	1000.000000
mean	500.500000	44.674000	18.97300	74.391290	1404.364060
std	288.819436	9.797741	18.89257	25.712083	1571.755048
min	1.000000	12.000000	0.00000	30.000000	0.000000
25%	250.750000	38.000000	5.00000	52.357500	345.217500
50%	500.500000	45.000000	13.00000	74.060000	872.870000
75%	750.250000	51.000000	26.00000	96.102500	1900.175000
max	1000.000000	83.000000	122.00000	119.960000	12416.250000

In [19]:
```python
numeric_columns_data = df.select_dtypes(include=["number"])
```

In [20]:
```python
numeric_columns_data
```

Out[20]:

	CustomerID	Age	Tenure	MonthlyCharges	TotalCharges
0	1	49	4	88.35	353.40
1	2	43	0	36.67	0.00
2	3	51	2	63.79	127.58
3	4	60	8	102.34	818.72
4	5	42	32	69.01	2208.32
...	...	...	...	...	...
995	996	42	41	37.14	1522.74
996	997	62	9	80.93	728.37
997	998	51	15	111.72	1675.80
998	999	39	68	65.67	4465.56
999	1000	50	1	56.67	56.67

1000 rows × 5 columns

In [21]:
```python
numeric_columns_data.corr()
```

Out[21]:

	CustomerID	Age	Tenure	MonthlyCharges	TotalCharges
CustomerID	1.000000	0.036730	-0.018585	-0.030504	-0.027490
Age	0.036730	1.000000	0.000472	0.006362	-0.001896
Tenure	-0.018585	0.000472	1.000000	-0.014552	0.894868
MonthlyCharges	-0.030504	0.006362	-0.014552	1.000000	0.304893
TotalCharges	-0.027490	-0.001896	0.894868	0.304893	1.000000

In [22]:
```python
df
```

Out[22]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month		2208.32	No	Yes
...	...	...	...	...	...	...	...	...	...	...
995	996	42	Male	41	37.14	Month-to-Month	Fiber Optic	1522.74	Yes	Yes
996	997	62	Male	9	80.93	Month-to-Month		728.37	No	Yes
997	998	51	Female	15	111.72	Month-to-Month	Fiber Optic	1675.80	No	Yes
998	999	39	Male	68	65.67	One-Year		4465.56	No	Yes
999	1000	50	Male	1	56.67	Month-to-Month		56.67	No	Yes

1000 rows × 10 columns

In [23]:
```python
df["Churn"].value_counts()
```

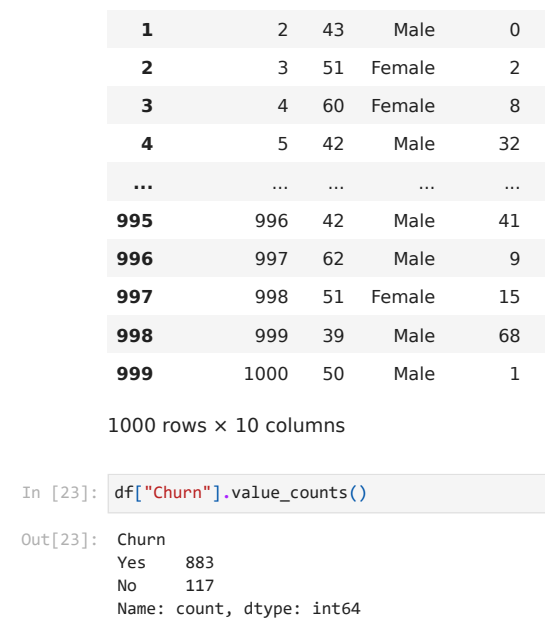
Out[23]:
```
Churn
Yes    883
No     117
Name: count, dtype: int64
```

In [24]:
```python
type(df["Churn"].value_counts())
```

Out[24]:
```
pandas.core.series.Series
```

In [25]:
```python
import matplotlib.pyplot as plt
```

In [26]:
```python
df["Churn"].value_counts().plot(kind="pie")
plt.title("Churn (yes/no)")
plt.ylabel("")
plt.show()
```



In [27]:
```python
df.head(2)
```

Out[27]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.4	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.0	Yes	Yes

In [28]:
```python
df.groupby("Churn")["MonthlyCharges"].mean()
```

Out[28]:
```
Churn
No     62.54641
Yes    75.96077
Name: MonthlyCharges, dtype: float64
```

In [29]:
```python
df.groupby(["Churn", "Gender"])["MonthlyCharges"].mean()
```

Out[29]:
```
Churn  Gender
No     Female    65.091912
       Male      59.013878
Yes    Female    74.975064
       Male      77.082518
Name: MonthlyCharges, dtype: float64
```

In [30]:
```python
df.groupby("Churn")["Tenure"].mean()
```

Out[30]:
```
Churn
No     30.264957
Yes    17.476784
Name: Tenure, dtype: float64
```

In [31]:
```python
df.groupby(["Churn", "Gender"])["Tenure"].mean()
```

Out[31]:
```
Churn  Gender
No     Female    30.132353
       Male      30.448980
Yes    Female    17.048936
       Male      17.963680
Name: Tenure, dtype: float64
```

In [32]:
```python
df.head(2)
```

Out[32]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.4	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.0	Yes	Yes

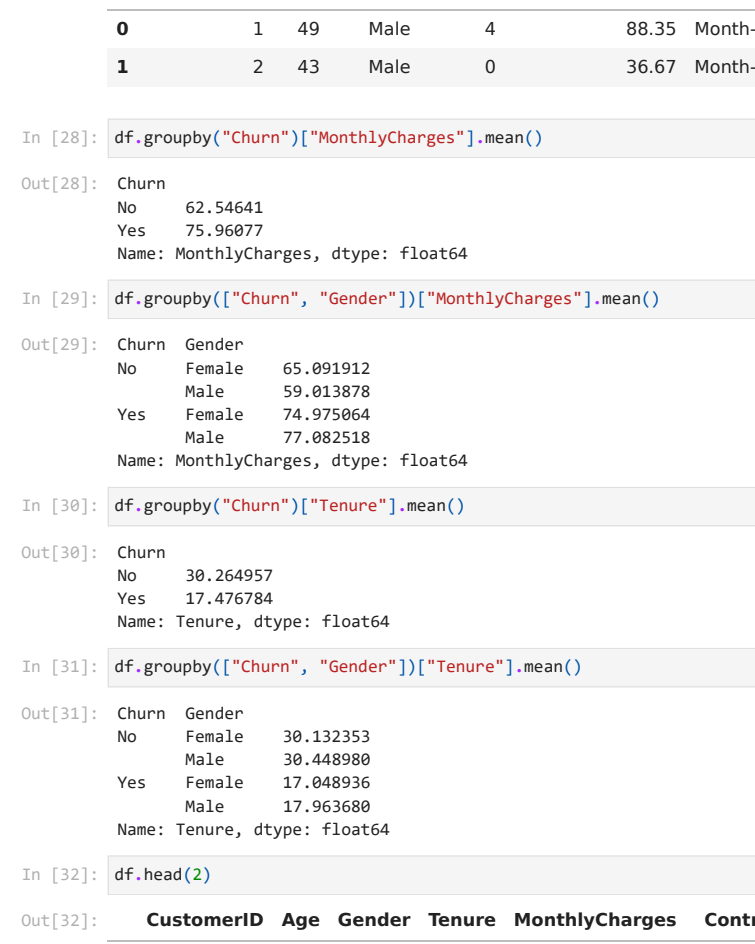
In [33]:
```python
df.groupby("Churn")["Age"].mean()
```

Out[33]:
```
Churn
No     43.487179
Yes    44.831257
Name: Age, dtype: float64
```

In [34]:
```python
df.groupby("ContractType")["MonthlyCharges"].mean()
```

Out[34]:
```
ContractType
Month-to-Month    75.909198
One-Year          73.822803
Two-Year          71.334500
Name: MonthlyCharges, dtype: float64
```

In [35]:
```python
df.groupby("ContractType")["MonthlyCharges"].mean().plot(kind="bar")
plt.title("Contract Type average")
plt.xlabel("Contract Type")
plt.ylabel("Mean")
plt.show()
```



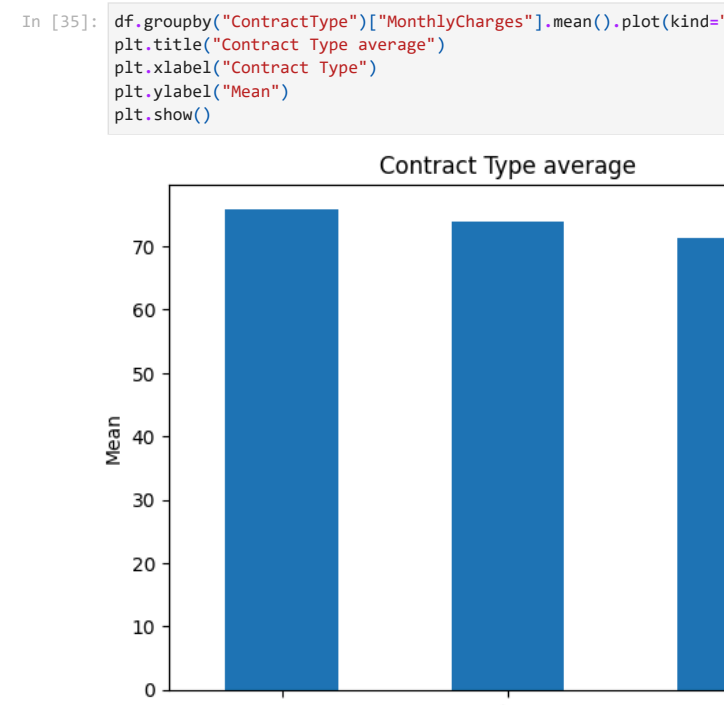
In [36]:
```python
df.head()
```

Out[36]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month		2208.32	No	Yes

In [37]:
```python
import seaborn as sns
```

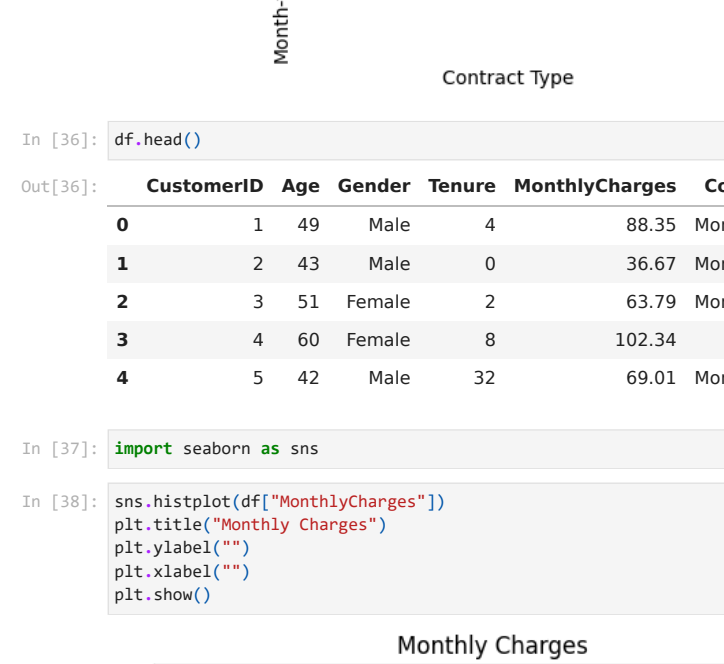
In [38]:
```python
sns.histplot(df["MonthlyCharges"])
plt.title("Monthly Charges")
plt.ylabel("")
plt.xlabel("")
plt.show()
```



In [39]:
```python
df.columns
```

Out[39]:
```
Index(['CustomerID', 'Age', 'Gender', 'Tenure', 'MonthlyCharges',
       'ContractType', 'InternetService', 'TotalCharges', 'TechSupport',
       'Churn'],
      dtype='object')
```

In [40]:
```python
sns.histplot(df["Tenure"])
plt.title("Histogram of Tenure")
plt.ylabel("")
plt.xlabel("")
plt.show()
```



In [41]:
```python
df.head()
```

Out[41]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.40	Yes	Yes
1	2	43	Male	0	36.67	Month-to-Month	Fiber Optic	0.00	Yes	Yes
2	3	51	Female	2	63.79	Month-to-Month	Fiber Optic	127.58	No	Yes
3	4	60	Female	8	102.34	One-Year	DSL	818.72	Yes	Yes
4	5	42	Male	32	69.01	Month-to-Month		2208.32	No	Yes

In [42]:
```python
df.columns
```

Out[42]:
```
Index(['CustomerID', 'Age', 'Gender', 'Tenure', 'MonthlyCharges',
       'ContractType', 'InternetService', 'TotalCharges', 'TechSupport',
       'Churn'],
      dtype='object')
```

In [43]:
```python
df.head(1)
```

Out[43]:

	CustomerID	Age	Gender	Tenure	MonthlyCharges	ContractType	InternetService	TotalCharges	TechSupport	Churn
0	1	49	Male	4	88.35	Month-to-Month	Fiber Optic	353.4	Yes	Yes

In [45]:
```python
y = df[["Churn"]]
X = df[["Age", "Gender", "Tenure", "MonthlyCharges"]]
```

In [49]:
```python
X["Gender"] = X["Gender"].apply(lambda x: 1 if x == "Female" else 0)
```

```
C:\Users\sahil\AppData\Local\Temp\ipykernel_11840\2388402656.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  X["Gender"] = X["Gender"].apply(lambda x: 1 if x == "Female" else 0)
```

In [50]:
```python
X # 1 female 0  male
```

Out[50]:

	Age	Gender	Tenure	MonthlyCharges
0	49	0	4	88.35
1	43	0	0	36.67
2	51	1	2	63.79
3	60	1	8	102.34
4	42	0	32	69.01
...	...	...	...	...
995	42	0	41	37.14
996	62	0	9	80.93
997	51	1	15	111.72
998	39	0	68	65.67
999	50	0	1	56.67

1000 rows × 4 columns

In [51]:
```python
type(X["Gender"][0])
```

Out[51]:
```
numpy.int64
```

In [47]:
```python
y
```

Out[47]:

	Churn
0	Yes
1	Yes
2	Yes
3	Yes
4	Yes
...	...
995	Yes
996	Yes
997	Yes
998	Yes
999	Yes

1000 rows × 1 columns

In [52]:
```python
y.head(2)
```

Out[52]:

	Churn
0	Yes
1	Yes

In [54]:
```python
y["Churn"] = y["Churn"].apply(lambda x: 1 if x == "Yes" else 0)
```

```
C:\Users\sahil\AppData\Local\Temp\ipykernel_11840\3691775968.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  y["Churn"] = y["Churn"].apply(lambda x: 1 if x == "Yes" else 0)
```

In [55]:
```python
y
```

Out[55]:

	Churn
0	1
1	1
2	1
3	1
4	1
...	...
995	1
996	1
997	1
998	1
999	1

1000 rows × 1 columns

In [57]:
```python
type(y["Churn"][0])
```

Out[57]:
```
numpy.int64
```

In [58]:
```python
from sklearn.model_selection import train_test_split
```

In [59]:
```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

In [60]:
```python
X_train
```

Out[60]:

	Age	Gender	Tenure	MonthlyCharges
638	32	1	24	57.65
442	24	0	65	97.85
147	31	0	35	119.61
118	56	1	14	100.54
637	45	1	15	50.04
...	...	...	...	...
8	40	1	53	49.81
222	51	0	23	36.39
33	34	1	1	90.40
805	41	1	27	68.38
517	35	0	3	85.26

800 rows × 4 columns

In [61]:
```python
y_train
```

Out[61]:

	Churn
638	1
442	1
147	1
118	1
637	1
...	...
8	0
222	1
33	1
805	1
517	1

800 rows × 1 columns

In [62]:
```python
X
```

Out[62]:

	Age	Gender	Tenure	MonthlyCharges
0	49	0	4	88.35
1	43	0	0	36.67
2	51	1	2	63.79
3	60	1	8	102.34
4	42	0	32	69.01
...	...	...	...	...
995	42	0	41	37.14
996	62	0	9	80.93
997	51	1	15	111.72
998	39	0	68	65.67
999	50	0	1	56.67

1000 rows × 4 columns

In [63]:
```python
from sklearn.preprocessing import StandardScaler
```

In [64]:
```python
scaler = StandardScaler()
```

In [65]:
```python
X_train = scaler.fit_transform(X_train)
```

In [66]:
```python
import joblib
```

In [67]:
```python
joblib.dump(scaler, "scaler.pkl")
```

Out[67]:
```
['scaler.pkl']
```

In [69]:
```python
X_test = scaler.fit_transform(X_test)
```

In [70]:
```python
X_train
```

Out[70]:
```
array([[-1.32212969,  0.94169658,  0.29663925, -0.64916071],
       [-2.15186395, -1.06191317,  2.52670282,  0.89375965],
       [-1.42584647, -1.06191317,  0.89494899,  1.72893246],
       ...,
       [-1.11469612,  0.94169658, -0.95437202,  0.60782043],
       [-0.38867864,  0.94169658,  0.45981463, -0.23733147],
       [-1.01097934, -1.06191317, -0.84558844,  0.41054155]],
      shape=(800, 4))
```

In [71]:
```python
X_test
```

Out[71]:
```
array([[ 1.31335092, -1.15133896, -0.66189172,  0.99561724],
       [-1.29020818,  0.86855395, -0.37158833, -1.55911651],
       [-0.51878326, -1.15133896,  2.24114212,  1.83439258],
       [ 0.92763847, -1.15133896, -0.12966884,  0.26669628],
       [-0.1330708 , -1.15133896,  0.16063454, -1.23987804],
       [ 1.50620715, -1.15133896,  0.30578623, -1.24565985],
       [-0.61521137, -1.15133896, -0.71027561,  1.60890202],
       [ 0.54192601,  0.86855395,  0.64447351, -0.53945314],
       [-0.51878326, -1.15133896, -0.66189172,  0.67761773],
       [ 0.34906978, -1.15133896,  0.06386674, -0.79798261],
       [ 1.12049469,  0.86855395, -0.27482054,  0.54587507],
       [-0.42235514,  0.86855395, -0.75865951, -1.37244666],
       [-1.19378006,  0.86855395, -0.80704341, -1.59339438],
       [ 0.05978543,  0.86855395,  1.12831249, -0.53325834],
       [ 1.21692281, -1.15133896, -0.61350782,  0.49672969],
       [ 1.12049469, -1.15133896, -0.37158833,  0.78623315],
       [-0.90449572, -1.15133896, -0.61350782, -1.33362595],
       [-0.71163949,  0.86855395,  0.16063454, -1.65988519],
       [-2.35091744,  0.86855395, -0.27482054, -0.03024093],
       [ 0.05978543, -1.15133896,  1.51538366,  1.68158762],
       [ 0.05978543,  0.86855395,  1.32184807,  0.46121287],
       [ 1.50620715,  0.86855395, -0.61350782,  1.18270007],
       [ 0.05978543,  0.86855395,  3.25720397, -0.907424  ],
       [ 1.02406658, -1.15133896,  2.77336499, -1.07385751],
       [-0.22949891,  0.86855395,  1.90245484, -0.28918339],
       [ 0.44549789,  0.86855395, -0.37158833, -0.54812585],
       [-1.48306441, -1.15133896, -1.000579  , -0.25077565],
       [-0.51878326,  0.86855395, -0.56512392,  0.61030095],
       [ 0.05978543,  0.86855395,  0.45093792,  1.80465756],
       [-1.77234875,  0.86855395, -0.46835613, -1.41126738],
       [-0.61521137,  0.86855395,  4.90225647, -0.17024331],
       [ 0.54192601, -1.15133896, -0.61350782, -0.03065391],
       [ 0.25264166,  0.86855395,  1.12831249, -1.22211962],
       [-0.1330708 ,  0.86855395, -1.000579  ,  1.24671296],
       [ 1.60263527,  0.86855395,  0.69285741, -0.80789428],
       [ 0.92763847, -1.15133896, -0.56512392, -0.79591768],
       [-1.00092383,  0.86855395, -0.9521951 ,  0.20350936],
       [-0.1330708 ,  0.86855395,  0.35417013,  0.75236826],
       [-1.19378006, -1.15133896, -0.51674002, -1.6140437 ],
       [ 0.34906978,  0.86855395,  0.98316079,  1.25951554],
       [ 3.7240538 , -1.15133896, -0.9038112 ,  0.50953227],
       [ 0.63835412, -1.15133896, -0.08128495,  0.80481753],
       [-0.42235514,  0.86855395, -0.9521951 ,  0.54587507],
       [-1.38663629,  0.86855395,  0.45093792, -1.64914754],
       [-1.29020818, -1.15133896, -0.12966884,  0.64333986],
       [ 1.69906338,  0.86855395,  2.53144551, -1.51162308],
       [ 1.7954915 ,  0.86855395, -0.61350782,  0.57767502],
       [ 0.63835412, -1.15133896, -1.000579  , -0.90701101],
       [-0.61521137, -1.15133896,  1.17669638, -1.42035308],
       [-1.38663629, -1.15133896, -0.75865951,  0.05483427],
       [ 0.15621355,  0.86855395,  1.41861587, -0.94459277],
       [ 0.63835412,  0.86855395,  0.16063454, -0.59933616],
       [-0.22949891, -1.15133896,  0.16063454, -0.9710239 ],
       [ 0.92763847,  0.86855395, -0.66189172, -0.13596544],
       [-0.42235514,  0.86855395, -0.80704341,  1.46353081],
       [ 1.21692281,  0.86855395, -0.75865951, -0.70506067],
       [ 0.63835412,  0.86855395,  2.48306161,  0.10604458],
       [ 0.15621355, -1.15133896, -1.000579  , -0.6922581 ],
       [ 2.4704883 ,  0.86855395, -0.9521951 ,  1.50276452],
       [-1.57949252,  0.86855395,  0.01548285, -1.13126262],
       [ 0.05978543, -1.15133896,  1.51538366,  1.39993091],
       [-0.8080676 ,  0.86855395, -1.000579  , -0.10127459],
       [ 0.44549789,  0.86855395,  0.59608961,  0.5359634 ],
       [ 0.05978543,  0.86855395, -0.32320443,  0.64457882],
       [-0.32592703, -1.15133896,  0.54770572, -1.02016928],
       [ 0.05978543, -1.15133896, -0.51674002,  1.18517799],
       [-0.22949891,  0.86855395, -0.9521951 , -0.11738105],
       [ 0.34906978,  0.86855395, -0.80704341,  0.85313694],
       [ 0.15621355,  0.86855395, -0.71027561,  0.88989273],
       [ 1.69906338,  0.86855395, -0.9521951 ,  1.40901661],
       [ 0.25264166, -1.15133896, -0.56512392, -0.95409146],
       [ 1.02406658,  0.86855395,  0.16063454,  1.12157809],
       [-0.32592703, -1.15133896, -0.61350782, -1.05320819],
       [-0.61521137,  0.86855395, -1.000579  ,  1.245474  ],
       [-0.61521137, -1.15133896, -0.9521951 ,  0.64623076],
       [ 0.34906978,  0.86855395, -0.41997223,  0.91962775],
       [ 0.83121035, -1.15133896,  2.09599043,  1.04434963],
       [ 0.73478224, -1.15133896,  0.886393  ,  1.7761615 ],
       [ 1.02406658, -1.15133896, -0.51674002, -0.09838368],
       [ 0.73478224,  0.86855395, -0.9038112 ,  0.942342  ],
       [-0.51878326, -1.15133896,  0.16063454, -0.31726646],
       [ 0.34906978,  0.86855395,  0.74124131, -0.30611583],
       [ 2.37406019, -1.15133896, -0.27482054,  1.28594667],
       [ 1.98834773, -1.15133896, -0.03290105, -0.54110508],
       [ 0.63835412, -1.15133896, -0.41997223,  0.0903511 ],
       [-0.71163949, -1.15133896, -0.80704341,  0.32864424],
       [-0.03664268, -1.15133896,  1.12831249,  0.90930309],
       [-0.42235514,  0.86855395, -0.71027561, -0.71455936],
       [-1.86877687,  0.86855395, -0.61350782, -0.07938631],
       [ 0.54192601, -1.15133896,  0.25740233,  0.76847473],
       [ 0.54192601,  0.86855395, -0.46835613,  1.39084521],
       [ 0.83121035,  0.86855395, -0.9521951 ,  1.50994526],
       [-0.03664268,  0.86855395,  0.20901844, -0.34369759],
       [ 0.54192601, -1.15133896,  0.9347769 ,  1.48872298],
       [-0.42235514, -1.15133896, -0.37158833, -0.48576491],
       [-0.42235514,  0.86855395,  0.20901844, -1.24194297],
       [ 1.12049469, -1.15133896,  0.54770572,  1.08317035],
       [ 0.44549789,  0.86855395,  1.03154469, -1.22418455],
       [-0.22949891,  0.86855395, -0.71027561,  0.43684667],
       [-0.71163949, -1.15133896, -0.32320443,  0.85644083],
       [-0.03664268,  0.86855395, -0.12966884,  1.03980678],
       [ 0.44549789,  0.86855395, -0.17805274,  0.97207702],
       [-0.51878326, -1.15133896, -0.12966884,  0.26339239],
       [-1.19378006,  0.86855395,  0.01548285, -1.02223421],
       [-1.38663629, -1.15133896,  0.7896252 , -0.59025046],
       [ 0.83121035,  0.86855395, -1.000579  , -0.30611583],
       [-1.96520498, -1.15133896, -0.22643664, -1.2502027 ],
       [ 0.54192601,  0.86855395,  2.82174889, -1.55498665],
       [ 2.18120396, -1.15133896, -0.75865951, -1.27209097],
       [ 2.4704883 , -1.15133896,  0.01548285,  0.36085717],
       [-0.51878326,  0.86855395, -0.9038112 , -1.5017114 ],
       [-1.67592064, -1.15133896, -0.66189172,  1.64689676],
       [-0.61521137, -1.15133896, -1.000579  ,  0.5603296 ],
       [ 0.05978543, -1.15133896, -0.32320443,  0.34268577],
       [-1.48306441,  0.86855395, -0.8554273 ,  0.47608037],
       [ 0.83121035, -1.15133896, -0.80704341,  0.51944394],
       [ 0.73478224,  0.86855395,  0.64447351,  0.35755328],
       [-1.96520498,  0.86855395, -0.71027561, -0.59231539],
       [-0.71163949, -1.15133896, -0.56512392,  1.25910255],
       [-0.8080676 ,  0.86855395,  0.49932182,  0.18698991],
       [-0.42235514, -1.15133896, -0.32320443, -1.51946982],
       [-0.71163949, -1.15133896, -0.71027561,  0.98570557],
       [-0.03664268, -1.15133896, -0.17805274,  0.66151126],
       [ 1.02406658, -1.15133896, -0.71027561,  1.86908344],
       [ 0.83121035, -1.15133896, -0.08128495,  0.55867765],
       [-0.03664268, -1.15133896, -0.9521951 ,  1.44246851],
       [ 0.54192601,  0.86855395, -0.8554273 , -1.49427765],
       [ 0.25264166, -1.15133896,  0.35417013, -1.65575532],
       [-1.86877687,  0.86855395, -0.71027561,  1.08688723],
       [-1.19378006, -1.15133896, -1.000579  , -0.98258752],
       [-0.51878326,  0.86855395, -0.32320443, -0.34741447],
       [-0.22949891, -1.15133896,  1.99922264,  0.98942245],
       [ 0.15621355,  0.86855395, -0.66189172,  1.31774662],
       [-0.22949891, -1.15133896, -0.41997223, -1.61776057],
       [-0.1330708 ,  0.86855395, -0.9038112 , -1.48147507],
       [-1.38663629, -1.15133896, -0.56512392,  1.81456923],
       [-0.61521137, -1.15133896,  2.82174889, -1.76478373],
       [-0.42235514,  0.86855395,  0.59608961, -0.58983747],
       [ 0.63835412,  0.86855395, -0.9038112 , -0.40936243],
       [-0.90449572, -1.15133896,  0.35417013,  1.17857021],
       [ 1.02406658,  0.86855395,  0.11225064, -0.66830489],
       [-1.19378006,  0.86855395, -0.8554273 ,  0.22622361],
       [-1.19378006, -1.15133896, -0.75865951, -1.63758392],
       [-0.8080676 , -1.15133896,  0.64447351, -0.43579355],
       [ 0.44549789,  0.86855395,  0.16063454, -1.4562829 ],
       [-0.51878326,  0.86855395,  0.16063454, -1.03338485],
       [ 0.83121035,  0.86855395, -0.75865951, -1.41663621],
       [-0.32592703,  0.86855395, -1.000579  , -1.48849584],
       [-1.00092383,  0.86855395,  0.45093792, -1.01521345],
       [ 0.34906978,  0.86855395, -0.8554273 ,  1.05178339],
       [ 0.34906978, -1.15133896,  0.25740233,  1.48872298],
       [ 0.05978543, -1.15133896, -0.32320443, -0.26894706],
       [-0.03664268, -1.15133896, -0.8554273 , -0.51302201],
       [-1.48306441, -1.15133896,  2.82174889,  0.39885192],
       [-0.51878326,  0.86855395,  1.03154469,  1.21491301],
       [-1.38663629, -1.15133896,  0.30578623,  1.49987361],
       [ 0.54192601, -1.15133896,  0.01548285, -0.96276418],
       [ 0.25264166, -1.15133896, -0.66189172,  0.23530932],
       [ 0.15621355,  0.86855395, -0.56512392,  1.0150276 ],
       [ 0.44549789,  0.86855395,  1.07992859,  0.37655066],
       [ 0.05978543, -1.15133896, -0.32320443, -0.83969423],
       [ 1.50620715,  0.86855395, -0.9521951 ,  0.78871107],
       [ 2.95262888,  0.86855395, -0.27482054,  0.64333986],
       [ 0.15621355,  0.86855395,  0.20901844,  0.7573241 ],
       [-0.22949891,  0.86855395, -0.61350782, -1.69829292],
       [ 0.73478224,  0.86855395, -0.08128495,  0.76310591],
       [-1.29020818, -1.15133896,  2.04760653,  0.05937712],
       [-0.8080676 ,  0.86855395, -0.51674002,  0.60617108],
       [-1.00092383, -1.15133896, -0.56512392, -0.32593918],
       [ 0.25264166,  0.86855395,  0.8380091 ,  0.91384594],
       [ 0.05978543,  0.86855395, -0.9038112 , -0.15826671],
       [ 0.54192601,  0.86855395, -0.61350782, -1.20312225],
       [-0.1330708 ,  0.86855395,  0.9347769 , -0.1615706 ],
       [-0.42235514,  0.86855395, -0.32320443,  0.46782065],
       [-0.8080676 ,  0.86855395,  0.74124131,  1.15337804],
       [-0.22949891,  0.86855395, -0.71027561, -1.23368324],
       [ 0.34906978,  0.86855395, -0.9038112 , -0.59851019],
       [-1.48306441, -1.15133896, -0.9038112 ,  0.28941053],
       [ 0.25264166, -1.15133896, -0.46835613,  1.49615674],
       [-0.22949891,  0.86855395,  1.90245484, -1.16967035],
       [-0.22949891, -1.15133896, -0.37158833, -0.87232016],
       [ 0.05978543,  0.86855395,  1.99922264,  0.51696603],
       [-0.8080676 ,  0.86855395,  0.01548285,  1.09266904],
       [-0.22949891,  0.86855395, -0.12966884,  0.70900469],
       [ 1.12049469,  0.86855395, -0.41997223, -0.24664579],
       [-1.67592064, -1.15133896,  0.06386674,  0.85809278],
       [-0.1330708 ,  0.86855395,  1.51538366, -0.91155386],
       [-0.03664268, -1.15133896, -0.27482054,  0.98735751],
       [-0.03664268,  0.86855395,  0.45093792, -0.05419414],
       [ 0.54192601, -1.15133896,  0.35417013,  0.70033198],
       [ 0.15621355,  0.86855395,  0.11225064, -0.93592006],
       [-0.8080676 , -1.15133896, -0.71027561, -0.76040085],
       [ 0.73478224, -1.15133896,  1.99922264,  0.51448811],
       [-1.29020818,  0.86855395,  0.69285741, -0.73975153],
       [-2.54377367,  0.86855395,  0.8380091 , -1.76271879],
       [-0.61521137,  0.86855395, -0.9521951 ,  0.39967789],
       [ 0.73478224, -1.15133896, -0.71027561, -0.42133903],
       [-0.51878326,  0.86855395, -0.75865951, -0.63113611],
       [ 1.60263527,  0.86855395, -0.9521951 , -0.14463816],
       [ 2.27763207,  0.86855395, -1.000579  , -1.48849584]])
```

In [128…
```python
from sklearn.metrics import accuracy_score
def modelperformance(predictions):
    print("Accuracy score on model is {}".format(accuracy_score(y_test, predictions)))
```

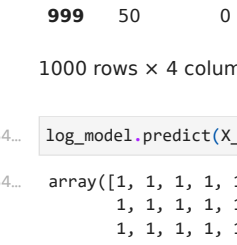
In [129…
```python
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
```

In [130…
```python
log_model = LogisticRegression()
```

In [131…
```python
import warnings
warnings.filterwarnings("ignore")
```

In [132…
```python
log_model.fit(X_train, y_train)
```

Out[132…



In [133…
```python
X
```

Out[133…

	Age	Gender	Tenure	MonthlyCharges
0	49	0	4	88.35
1	43	0	0	36.67
2	51	1	2	63.79
3	60	1	8	102.34
4	42	0	32	69.01
...	...	...	...	...
995	42	0	41	37.14
996	62	0	9	80.93
997	51	1	15	111.72
998	39	0	68	65.67
999	50	0	1	56.67

1000 rows × 4 columns

In [134…
```python
log_model.predict(X_test)
```

Out[134…
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1])
```

In [135…
```python
log_model.predict([[40,0,20,80]])[0]
```

Out[135…
```
np.int64(1)
```

In [136…
```python
y_pred = log_model.predict(X_test)
```

In [137…
```python
modelperformance(y_pred)
```

```
Accuracy score on model is 0.85
```

In [138…
```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

def evaluate_model(y_test, y_pred):
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

In [139…
```python
evaluate_model(y_test, y_pred)
```

```
Accuracy: 0.85

Classification Report:
               precision    recall  f1-score   support

           0       0.50      0.03      0.06        30
           1       0.85      0.99      0.92       170

    accuracy                           0.85       200
   macro avg       0.68      0.51      0.49       200
weighted avg       0.80      0.85      0.79       200


Confusion Matrix:
 [[  1  29]
 [  1 169]]
```

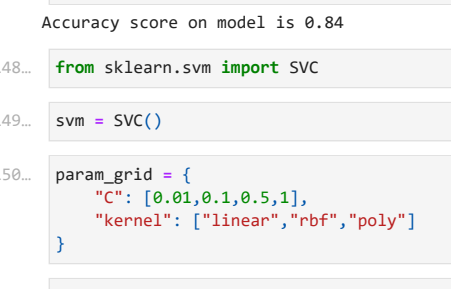
In [140…
```python
from sklearn.neighbors import KNeighborsClassifier
```

In [141…
```python
param_grid = {
    "n_neighbors": [3,5,7,9],
    "weights": ["uniform", "distance"],
}
```

In [142…
```python
gridkn = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
```

In [143…
```python
gridkn.fit(X_train,y_train)
```

Out[143…



In [144…
```python
gridkn.best_params_
```

Out[144…
```
{'n_neighbors': 9, 'weights': 'uniform'}
```

In [145…
```python
gridkn.predict(X_test)
```

Out[145…
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1])
```

In [146…
```python
y_pred = gridkn.predict(X_test)
```

In [147…
```python
modelperformance(y_pred)
```

```
Accuracy score on model is 0.84
```

In [148…
```python
from sklearn.svm import SVC
```

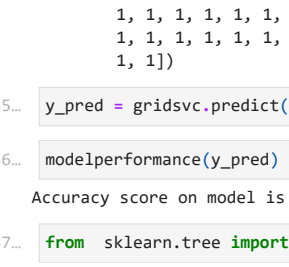
In [149…
```python
svm = SVC()
```

In [150…
```python
param_grid = {
    "C": [0.01,0.1,0.5,1],
    "kernel": ["linear","rbf","poly"]
}
```

In [151…
```python
gridsvc  = GridSearchCV(svm, param_grid, cv = 5)
```

In [152…
```python
gridsvc.fit(X_train, y_train)
```

Out[152…



In [153…
```python
gridsvc.best_params_
```

Out[153…
```
{'C': 0.01, 'kernel': 'linear'}
```

In [154…
```python
gridsvc.predict(X_test)
```

Out[154…
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1])
```

In [155…
```python
y_pred = gridsvc.predict(X_test)
```

In [156…
```python
modelperformance(y_pred)
```

```
Accuracy score on model is 0.85
```

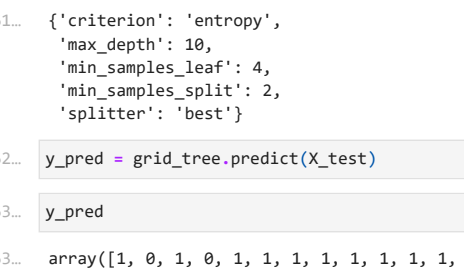
In [157…
```python
from  sklearn.tree import DecisionTreeClassifier
```

In [158…
```python
param_grid = {
    "criterion":["gini","entropy"],
    "splitter": ["best","random"],
    "max_depth": [None,  10, 20, 30],
    "min_samples_split": [2,5,10],
    "min_samples_leaf":[1,2,4]
}
```

In [159…
```python
grid_tree = GridSearchCV(DecisionTreeClassifier(), param_grid, cv = 6)
```

In [160…
```python
grid_tree.fit(X_train, y_train)
```

Out[160…


▸ GridSearchCV ⓘ ?
▸ best_estimator_: DecisionTreeClassifier
▸ DecisionTreeClassifier ?

In [161…
```python
grid_tree.best_params_
```

Out[161…
```
{'criterion': 'entropy',
 'max_depth': 10,
 'min_samples_leaf': 4,
 'min_samples_split': 2,
 'splitter': 'best'}
```

In [162…
```python
y_pred = grid_tree.predict(X_test)
```

In [163…
```python
y_pred
```

Out[163…
```
array([1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
       0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1])
```

In [164…
```python
modelperformance(y_pred)
```

```
Accuracy score on model is 0.81
```

In [165…
```python
from sklearn.ensemble import RandomForestClassifier
rfc_model = RandomForestClassifier
```

In [171…
```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
```

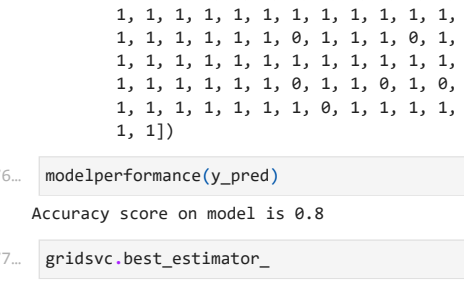
In [172…
```python
rf = RandomForestClassifier(random_state=42)

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10],
    "class_weight": ["balanced"]
}

grid_rfc = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

grid_rfc.fit(X_train, y_train)
```

Out[172…


▸ GridSearchCV ⓘ ?
▸ best_estimator_: RandomForestClassifier
▸ RandomForestClassifier ?

In [173…
```python
grid_rfc.best_params_
```

Out[173…
```
{'class_weight': 'balanced', 'max_depth': None, 'n_estimators': 100}
```

In [174…
```python
y_pred = grid_rfc.predict(X_test)
```

In [175…
```python
grid_rfc.predict(X_test)
```

Out[175…
```
array([1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1])
```

In [176…
```python
modelperformance(y_pred)
```

```
Accuracy score on model is 0.8
```

In [177…
```python
gridsvc.best_estimator_
```

Out[177…

▾ SVC ⓘ ?
▸ Parameters

In [178…
```python
best_model = gridsvc.best_estimator_
```

In [179…
```python
joblib.dump(best_model, "model.pkl")
```

Out[179…
```
['model.pkl']
```

In [181…
```python
X.columns
```

Out[181…
```
Index(['Age', 'Gender', 'Tenure', 'MonthlyCharges'], dtype='object')
```

In [183…
```python
import streamlit as st
import joblib
import numpy as np
```

In [ ]: